

CSE 220 Lab Test — Practice Question Bank (with Solutions)

Fourier Series & Continuous Fourier Transform

[!NOTE] All questions assume you have your completed `fs_redrawer.py` and `cft_edge_detector.py` from Offline 2. Each question is designed to be solvable in **20–25 minutes** by adding a new method or a short block of code to your existing classes.

Category A: Fourier Series (`fs_redrawer.py`)

Q1: Low-Pass Harmonic Truncation

In `FourierEpicycles`, implement a method:

```
def truncate_harmonics(self, max_n):
```

Given an integer `max_n`, **set to zero** all coefficients c_n where $|n| > \text{max_n}$, i.e., keep only harmonics $n = -\text{max_n}, \dots, \text{max_n}$.

In `--main--`, load `heart.svg` with $N = 150$. For `max_n ∈ {5, 15, 50, 150}`:

1. Truncate the harmonics.
2. Print the MSE between the original signal and the truncated reconstruction.
3. Save the comparison plot as `heart-truncated-{max_n}.png`.

Expected output format:

```
max_n | Harmonics Retained | MSE  -----+-----+-----  5 | 11 |  
... 15 | 31 | ... 50 | 101 | ... 150 | 301 | ...
```

► **Solution**

Q2: Phase Scrambling

In `FourierEpicycles`, implement:

```
def scramble_phases(self, seed=42):
```

For every coefficient c_n in `self.coeffs`, **keep its magnitude** $|c_n|$ unchanged but **replace its phase** $\arg(c_n)$ with a uniformly random angle from $[0, 2\pi)$. Use `np.random.default_rng(seed)` for reproducibility.

In `--main--`, load `heart.svg` with $N = 150$. Compute all coefficients, scramble the phases, then save the comparison plot. Also print the MSE.

Hint: A complex number with magnitude r and angle θ is $r \cdot e^{j\theta}$.

► **Solution**

Q3: Conjugate Symmetry Check

In `FourierEpicycles`, implement:

```
def check_conjugate_symmetry(self, tol=1e-9):
```

For a **real-valued** signal, the Fourier coefficients satisfy $c_{-n} = \overline{c_n}$ (conjugate symmetry). But the SVG signals are **complex-valued** ($x + jy$), so this property should NOT hold.

This method should:

1. For every n from 1 to N , compute $\delta_n = |c_{-n} - \overline{c_n}|$.
2. Return `(is_symmetric, max_delta)` where `is_symmetric = True` if all $\delta_n < \text{tol}$.

In `--main--`, load `heart.svg` and `circle.svg`, compute coefficients, and print the result of this check for both.

► **Solution**

Q4: Harmonic Energy Spectrum Plot

In `FourierEpicycles`, implement:

```
def plot_energy_spectrum(self):
```

Plot a bar chart where the x-axis is the harmonic index n (from $-N$ to N) and the y-axis is the energy $|c_n|^2$ of each harmonic.

Also implement:

```
def cumulative_energy_ratio(self):
```

Sort harmonics by energy (descending). Return a list of `(count, ratio)` tuples showing, for each additional harmonic added, what fraction of total energy has been accumulated.

In `--main--`, print how many harmonics are needed to reach 90%, 95%, 99%, and 99.9% of total energy.

► Solution

Q5: Time-Shifted Reconstruction

In `FourierEpicycles`, implement:

```
def approximate_shifted(self, t, t_shift):
```

Instead of reconstructing $\hat{f}(t) = \sum c_n e^{jn\omega t}$, reconstruct a **time-shifted** version: $\hat{f}_{\text{shifted}}(t) = \sum_{n=-N}^N c_n \cdot e^{jn\omega(t-t_{\text{shift}})}$

This effectively starts drawing the shape from a different point on the curve.

In `--main--`, load `heart.svg`, and generate comparison plots for `t_shift ∈ {0, T/4, T/2, 3T/4}`.

► Solution

Q6: Derivative of the Fourier Series

In `FourierEpicycles`, implement:

```
def approximate_derivative(self, t):
```

The derivative of the Fourier Series reconstruction is: $\hat{f}'(t) = \sum_{n=-N}^N c_n \cdot (jn\omega) \cdot e^{jn\omega t}$

Each coefficient just gets multiplied by $j n \omega$. Return the derivative signal evaluated at time(s) `t`.

In `__main__`, plot the magnitude of the derivative $|\hat{f}'(t)|$ vs t (this represents the "speed" of the pen).

► Solution

Category B: 2D CFT / Image Filtering (`cft_edge_detector.py`)

Q7: Low-Pass Filter

In `FrequencyFilter`, implement:

```
def low_pass(self, real, imag, cutoff):
```

This is the **exact opposite** of the given `high_pass` filter. Zero out all frequency components **outside** the cutoff radius (i.e., where $d(i, j) > \text{cutoff}$), keeping only the low-frequency content near the center.

In `__main__`, run the pipeline with `cutoff = 15`. Save the reconstructed (blurred) image as `pikachu_lowpass.png`.

► Solution

Q8: Complementarity Verification (High-Pass + Low-Pass)

Implement:

```
def verify_hp_lp_complementarity(self, I_original, I_hp, I_lp):
```

in a `ReconstructionValidator` class.

Since `high_pass` and `low_pass` are exact complements, verify: $I_{\text{hp}}(x, y) + I_{\text{lp}}(x, y) \approx I_{\text{original}}(x, y)$

► Solution

Q9: Ring Filter (Annular Band-Pass)

In `FrequencyFilter`, implement:

```
def ring_filter(self, real, imag, r_inner, r_outer):
```

Retain only frequency components in the annular ring $r_{\text{inner}} < d(i, j) \leq r_{\text{outer}}$. Zero everything else.

In `__main__`, for rings `(0, 10)`, `(10, 30)`, and `(30, 50)`, apply the ring filter, reconstruct, and print the fraction of total spectral energy retained.

► Solution

Q10: Spectrum Energy Distribution

In `CFT2D`, implement:

```
def radial_energy_profile(self, real, imag, num_bins=50):
```

Divide the frequency plane into `num_bins` concentric annular rings of equal width. For each ring, compute the total energy.

► Solution

Q11: Frequency Spectrum Masking with a Custom Shape

In `FrequencyFilter`, implement:

```
def cross_mask(self, real, imag, width):
```

Zero out all frequency components in a **cross-shaped** region: entry (i, j) is zeroed if $|i - c_x| \leq \text{width}$ **or** $|j - c_y| \leq \text{width}$.

► Solution

Q12: DC Component Extraction

In `FrequencyFilter`, implement:

```
def extract_dc(self, real, imag):  
def remove_dc(self, real, imag):
```

► Solution

Q13: Spectrum Rotation

In `FrequencyFilter`, implement:

```
def rotate_spectrum_90(self, real, imag):
```

Rotate the entire frequency spectrum by 90° counterclockwise using `np.rot90`.

► Solution

Q14: Spectrum Scaling (Contrast Enhancement)

In `FrequencyFilter`, implement:

```
def scale_spectrum(self, real, imag, factor):
```

Multiply **all** entries of `real` and `imag` by `factor`, **except** the DC component (center pixel).

► Solution

Q15: Spectral Energy Thresholding (Denoising)

In `FrequencyFilter`, implement:

```
def threshold_spectrum(self, real, imag, threshold):
```

For every entry (i, j) , compute energy $e = \text{real}^2 + \text{imag}^2$. If $e < \text{threshold}$, zero both components.

► **Solution**

Quick Reference: Patterns You Should Know Cold

These are the building blocks that every question above uses. Make sure you can write these from memory:

Pattern	Code
Distance from center	<code>d = np.sqrt((i - cx)**2 + (j - cy)**2)</code>
Filter loop skeleton	<code>for i in range(rows): for j in range(cols): if condition: real[i,j] = 0; imag[i,j] = 0</code>
MSE (complex signals)	<code>np.mean(np.abs(f_true - f_approx)**2)</code>
MSE (real images)	<code>np.mean((I_true - I_approx)**2)</code>
Max absolute error	<code>np.max(np.abs(A - B))</code>
Energy of coefficient	<code>abs(cn)**2</code>
Energy of spectrum entry	<code>real[i,j]**2 + imag[i,j]**2</code>
Sort dict by value	<code>sorted(d.items(), key=lambda x: abs(x[1]), reverse=True)</code>
Modify only DC pixel	<code>real[rows//2, cols//2] += amount</code>
Clip and save image	<code>plt.imsave(path, np.clip(img, 0, 1), cmap='gray')</code>
Edge map pipeline	<code>edge = np.abs(raw); edge /= edge.max(); edge = 1 - edge</code>
Complex number from mag+phase	<code>magnitude * np.exp(1j * angle)</code>
Derivative coefficient	<code>cn * (1j * n * omega)</code>
Complementarity check	<code>delta = np.max(np.abs(A - (B + C))); valid = delta < 1e-9</code>